

Outside of the function, are the variables that handle the values for the function to be worked with, its derivative, and the max amount of iterations. Inside of the function findRoots, is all the code responsible for the Newton-Raphson method and all the graphing. The function itself is being called by another variable that also contains the before mentioned variables.

Function inputs

- func - Contains the original function that will be worked throughout the code.
- derivative - This one contains the derivative from that function; the user MUST enter it manually. There's a way for MATLAB to calculate the derivative, but it requires more work on my side and using stuff that I don't even know how to use.
- maxIters - It's an integer that contains "how many tries" the user has to find the root of a given function.
- minError - Float type variable; it handles what the minimum acceptable error will be accepted with respect to the given input. The default value if no input is actually given will be 0.1.
- initialGuess - Float type; handles the first guess or iteration from the user's input. If nothing is entered, a default value of 0 will be used (assuming that the guessed root = 0)

For both the minError and the initialGuess, the MATLAB function nargin is used, where if a minimum number of arguments with which a function is called, it will assign the default values

Function output

- root - Root guessed by the user; actually it's the last entered value by the user, if either it was guessed by the user, or the maxIters was reached.

When the code starts running, the following happens:

- These variables are initialized:
 - currentGuess - First iteration.
 - currentIter - Counter of iterations or tries.
 - guesses - A vector that will store each iteration for graphing them later.
- Initial information (that is actually defined in the very beginning of the code).
- The while loop begins. This is responsible for running n iterations or n times the code of the Newton-Raphson algorithm.

While loop (iteration loop)

- This loop runs the algorithm or method while the number of iterations are not exceeded. The code will exit this loop once either, a root is found, or the number of iterations are exceeded.
- It will ask, during n iterations, the user for the input on the guess of the root.

- It will evaluate the function and its derivative with the given input. This is done so that feedback is given to the user on whether the number is close to the actual root, or not. It also helps to build the tangential line. There's an if conditional that check if the result of the derivative is equal to zero, as this would be a math error.
- Once the root is found, or we reach the max number of iterations, the code will exit the while loop. It will display the final graph with all the guessed points, the last guessed root, and the real root that is obtained from the Newton-Raphson method.
 - Exits when maxIter is bigger than currentIter.
 - Exits if the minError obtained from the current iteration is bigger than the iterationError.
- The real root value that is obtained from the Newton-Raphson method is stored in a new variable, guess.

iterationError

- Measures how large the change from two iterations is..
 - If it's big, then we are not even close to the real value of the root.
 - If it's small, we are getting closer, or we are very close to the value of the root.
 - It's compared to the input of the user minError; if this is smaller than the lastly mentioned, then we have approximated or found the value for the root, and the code exits.

Tangential line

- The for loop is responsible for getting the tangential line.
- J represents the current iteration, and it will run through all the iterations that the user defined. For every j, the x part will run to define an interval where the tangent line will be drawn. This is helpful since the graphing occurs for every iteration (it gets updated), and a final version is displayed after the recently mentioned conditionals are met.
 - In x, it will first create 100 points or values around guesses(j-1). This creates the range for the tangential line, so that it is not in places it shouldn't be.
 - In y, it will apply the tangential formula using the variables that we have been using. The formula is: $y = f'(a)(x - a) + f(a)$, where $a = \text{guesses}(j)$
 - Using this data, the tangential line will be drawn using the plot function over the original graph. Simply, it gets updated for every iteration. Displayname actually displays a string that is calculated based on the current value for j; the true one for us is actually j-1, just like it happens in the variable guesses(j-1).

Graphs

- Every iteration is plotted as a point on the function graph (the graph is updated for every iteration).
 - The points mark in a clearer way the steps we take to slowly reach the real value of the root.
- When the loop finishes, or the root is found, a final graph is displayed that includes:
 - All tangent lines
 - The function $f(x)$
 - The estimated root (marked with a red star)

%d acts as a placeholder for a number in a variable.

Important:

- sprintf works for printing strings. just like f-string would work on python.
- %d in the sprintf is basically a, "hey print a number here", and the number is stored in a variable that is mentioned later. It's a position marker.
 - %d. %f. %s
- %.3 says how many decimals will be shown when something is displayed.
- currentGuess stores the actual value of x, before the next iteration
- nextGuess is the new value of x, from the iteration.
- nextGuess is stored in currentGuess.
- abs gets the absolute value of a given variable or number.